BOOX STORE

How the Cart Code Works

A friendly, code-by-code tour of the shopping basket we built.

CLICK → REMEMBER → SHOW

A book button sends its ID to the shared cart.
The cart remembers the quantity.
The page redraws with the new basket.

This little book explains what the files and functions mean, not just what buttons to press.

1. First, the big picture

Think of the cart like a small shop team. One part keeps the list, another shows the books, and buttons ask the list-keeper to make changes. React is the page builder: when the list changes, React draws the affected parts again.

File	Its job in the shop
App.jsx	Loads books from /api/books and wraps the page in CartProvider so parts can share a cart.
BookCard.jsx	Shows one book and its Add to cart button.
CartContext.jsx	Keeps the shared list, changes it, and saves it in this browser.
CartSection.jsx	Displays the basket, quantities, removals, and subtotal.
Header.jsx	Shows the total copy count and a link to the cart.
QuantityStepper.jsx	Reusable plus/minus control for changing a cart quantity.
cart-context.js	Defines the shared Context and the useCart hook.

A React component is a reusable piece of the page. A book card is one component; the cart is another. They look different but can share the same cart information.

App.jsx: loading the catalogue

```
useEffect(() => {
  fetch('/api/books').then(...);
}, [])
```

The real code checks for a failed response, reads the JSON book list, and tracks loading/error state. The empty [] means ?do this when this component first appears.? An AbortController cancels a request if the page component goes away.

App.jsx: sharing the cart

```
<CartProvider>
  <Header />
  <BookGrid books={books} />
  <CartSection books={books} />
</CartProvider>
```

CartProvider is like a shared noticeboard around the parts that need cart access. A component using the cart outside this wrapper gets a clear error instead of silently failing.

2. The cart?s memory and actions

A cart line stores a book ID and quantity, for example:

```
{ bookId: 2, quantity: 3 }
```

That means ?book number 2, three copies.? It does not save a second copy of the title, picture, or price. CartSection looks up the current book details from the catalogue.

React state: memory that redraws the page

```
const [cart, setCart] = useState(readStoredCart)
```

useState gives the cart memory. cart is the current remembered value; setCart changes it. When setCart runs, React knows it should redraw the parts that use the cart.

The four actions

Function	What it means
addItem(bookId)	If that book is already there, add one copy. Otherwise start it at one.
setItemQuantity(bookId, quantity)	Set the number. Zero removes the line; negative or fractional numbers are rejected.
removeItem(bookId)	Remove that book line entirely.
clearCart()	Replace the list with an empty list.
itemCount	Add all quantities together. Two copies plus one copy means three items.

Updating the list without changing the old one

```
items.map(item => item.bookId === bookId
  ? { ...item, quantity: item.quantity + 1 }
  : item)
```

map makes a new list, changing only the matching line. The ... copies its existing fields before replacing quantity. filter makes a new list with selected lines left out. This ?make a new list? style helps React notice changes reliably.

3. Saving, buttons, and the basket display

localStorage is a small notebook belonging to this browser on this device. It is not the online account. CartContext reads the note at startup with readStoredCart(), and saveItems() writes each update back.

Text to data and data to text

```
JSON.parse(savedText)       // text becomes a JavaScript value
JSON.stringify(items)       // JavaScript value becomes text
```

The saved note is checked: it must be a list with positive whole-number IDs and quantities, with no duplicate book IDs. If it is broken or blocked, the code logs a useful error and shows a message. The cart can still update on the page, but a change might not survive refresh.

From a book click to a shared update

```
onClick={() => addItem(book.id)}
```

onClick means ?when clicked.? This waits for the click and then calls addItem with that book?s ID. The book card does not keep a separate quantity; the shared cart does.

Plus and minus are reusable

```
<QuantityStepper
  quantity={item.quantity}
  onQuantityChange={number =>
    setItemQuantity(item.bookId, number)
  }
/>
```

quantity and onQuantityChange are props: information and a function passed into the stepper. The stepper asks the parent to change the value; it does not own the saved cart. This is called a controlled component. The minus button is disabled at zero.

Joining the cart line to its book

```
books.find(book => book.id === item.bookId)
```

find searches the catalogue for the matching ID. That gives the cart the current title, cover, and price. While the catalogue loads or has an error, the cart avoids incorrectly calling saved items unavailable. A missing ID after a successful load can be removed.

The header count comes from reduce(), which adds up each quantity. The number means copies, not different titles.

4. Money, boundaries, and a glossary

How the subtotal works

```
lineCents = round(price * 100) * quantity
subtotalCents = add every lineCents
display subtotalCents / 100 as USD
```

For example, $12.50 becomes 1,250 cents. Two copies cost 2,500 cents, or $25.00. Counting whole cents avoids many small decimal arithmetic surprises. Intl.NumberFormat prints the amount like money, such as $25.00.

What the cart can and cannot do

It can do now	It cannot do yet
Add, change, remove, and empty cart items.	Take payment or connect to Stripe.
Show catalogue prices and a books-only subtotal.	Calculate shipping, tax, inventory, or final checkout amount.
Remember the cart in this browser on this device.	Sync with the account or another device.

A safety lesson for checkout later

The browser is like paper a shopper can edit. Later, the server must check the IDs and quantities and calculate the real total itself. It must not trust a price sent by the browser. Stripe should confirm payment to the server before an order is marked paid. The current cart is a basket, not a checkout.

Mini glossary

Word	Plain meaning
State	A component?s remembered information.
Props	Information or a function passed from one component to another.
Context	A way for components to share information without passing it through every layer.
Render	Draw the current version of a component on the screen.
Callback	A function passed to another component so it can ask for an action.
localStorage	A small saved notebook inside one browser on one device.

The cart story: one shared list, a few clear actions, and page pieces that redraw when the list changes.